

Riverline Hiring Assignment -AI Engineer

Before You Begin

A note on AI tools. You may use any AI coding assistant, including Claude, Copilot, or GPT. We assume you will use Fable 5.1 and GPT-6 Astra. This changes what we're evaluating. We are not testing whether you can produce code. We are testing whether you understand what you built, can defend your decisions under live questioning, can modify your system under novel constraints on the spot, and made genuine engineering trade-offs that reflect judgment, not generation. Your submission will be followed by a live technical session with no AI assistance. If you cannot navigate your own codebase fluently, explain your statistical methodology, and adapt your system live, your submission quality is irrelevant.

General Guidelines

- Keep your code short and concise.
- Feel free to use any tools that help you solve this assignment.
- The assignment is designed to be hard and lengthy. Starting early will help.
- There are no right answers. Approach matters more than correctness.
- Make suitable assumptions wherever you feel things are not defined, since it is meant to be open-ended.
- The submission details are at the end of this document.
- Reach out to us if you have any queries (jayanth@riverline.ai)
- The deadline (mentioned on the email) will be strict, no extensions! So, plan accordingly.

Reference

Watch this Riverline product demo before starting:



Do you have enough money for your EMIs? I

Riverline



Watch on

Treat this merely as a reference for we expect:

- Real-time voice conversation.
- Useful cards generated during the conversation.
- A clear financial outcome for the user.

Do not copy the interface, questions or conversation flow.

Your product should solve the problem independently and improve upon the reference in meaningful ways that you could think of. We strongly advise not to just simply recreate the video.

Problem

Build a real-time voice assistant that helps a person plan their finances for the next 30 days.

The person may have:

- Multiple loans and credit-card payments.
- Income arriving on different dates.
- Essential household expenses.
- Optional expenses.
- Missing or uncertain information.

- Conflicting numbers.
- More payments than they can afford.

The assistant should understand the situation, ask the right questions, perform the calculations and produce a realistic plan.

The assistant must speak only in **English**.

What you must build

1. Web application

Build a simple web application where the user can:

- Start and end a voice conversation.
- Speak naturally with the assistant.
- See useful information generated during the conversation.
- Review the final financial plan.

2. Real-time voice agent

You must use:

- **Pipecat**
- **Daily**

The conversation must be real-time. It cannot work as a series of recorded voice messages.

The agent should:

- Ask questions based on the conversation.
- Avoid following a fixed questionnaire.
- Remember information already provided.
- Ask for clarification when information conflicts.
- Allow the user to correct previous information.

- Avoid presenting guesses as facts.
- Explain the final plan simply.
- Confirm that the user understands the plan.

3. Generative cards

Cards should appear or update during the conversation.

They could show:

- Income.
- Available money.
- Essential expenses.
- Loan and credit-card payments.
- Missing information.
- Upcoming payment dates.
- Monthly shortfall or surplus.
- Proposed actions.
- Final plan.

You decide which cards are useful and when they should appear.

Cards must remain consistent with the conversation. If the user corrects an amount, every affected card and calculation should update.

4. Financial plan

The final plan may:

- Show the user's shortfall or surplus.
- Prioritise upcoming payments.
- Suggest changes to optional expenses.
- Account for when income and payments occur.
- Build a practical 30-day plan.
- Clearly explain when the situation cannot be solved using the available money.

The assistant must not:

- Invent numbers.
- Promise loan or lender approval.
- Invent settlement or repayment offers.
- Recommend taking another loan.
- Claim that an action has been completed when it has not.

The calculations must be visible and testable.

Decision journal

A decision journal is mandatory.

It may be handwritten or typed, but it must be written entirely by you.

Do not use AI to:

- Write the journal.
- Rewrite the journal.
- Improve its language.
- Summarise your decisions.
- Paraphrase your notes.

AI-generated journal content will disqualify the submission. A missing journal will also disqualify the submission.

Grammar and presentation will not be evaluated.

For each important decision, briefly record:

- Approximate date and time.
- What happened.
- What you noticed.
- Options you considered.
- What you decided and why.

- What you tested.
- What changed your mind.
- AI suggestions you rejected or corrected.
- Limitations you accepted.

Write entries while working, not only after completing the assignment.

Submission

Submit your work using the [Riverline engineering assignment submission form](#).

Submit:

1. Source-code repository.
2. README with Docker setup, environment variables and the local web address.
3. Short product demo video.
4. Decision journal.
5. Test and evaluation results.
6. A short list of:
 - What works.
 - What does not work.
 - What you would build next.
 - Where AI helped.
 - Where AI produced incorrect or poor work.

Deployment is not required.

The complete application must start with one command:

```
docker compose up --build
```

After this command completes, the web application must be available at the local address written in your README. Starting the agent backend or web application must not require extra commands in separate terminals.

Include a `.env.example` file containing every required environment variable. The README must explain:

- What each variable is used for.
- Which variables are required.
- How the reviewer should provide API keys.
- The exact command to start the system.
- The exact local address to open.

Do not commit API keys or other secrets.

What we will evaluate

- Does the product work end to end?
- Does the agent ask useful questions?
- Are the calculations correct?
- Is the financial plan realistic?
- Does the system handle corrections and missing information?
- Do the voice experience and cards remain consistent?
- Did you understand the code you submitted?
- Did you make sensible trade-offs?
- Did you test the important parts?
- Is the implementation simple enough to understand and extend?

We care more about one strong, complete journey than many unfinished features.

Optional depth work

This section is not required. Candidates who complete it well will receive additional consideration.

If you attempt this section, choose **only one** track. Do not attempt both.

Track A: Conversational intelligence

Go deeper into the quality of the conversation.

You could focus on:

- Asking better follow-up questions.
- Handling corrections naturally.
- Handling interruptions.
- Avoiding repeated questions.
- Knowing when enough information has been collected.
- Explaining difficult outcomes clearly.
- Checking whether the user actually understands the plan.
- Adapting the conversation instead of following a script.

Show real conversations that demonstrate your work.

Track B: Evaluation and regression testing

Go deeper into testing the agent.

You could focus on:

- Creating different user scenarios.
- Measuring conversation quality.
- Checking financial calculations.
- Testing missing and conflicting information.
- Separating rule-based checks from LLM-based evaluation.
- Finding failures and turning them into permanent tests.
- Comparing agent versions.

- Explaining where your evaluation method may be wrong.

Show at least one failure you found, the change you made and evidence that the change improved the system.